

团队分工会议 — 2026-05-21

会前准备材料。预计时长：30-40分钟。

一、开场（5分钟）：我们在做什么

一句话总结

forhacker = AI 驱动的网络犯罪综合对抗平台

覆盖三个核心领域（电子取证、渗透测试、开源情报）+ CTF 竞赛作为实战检验场景 + 大数据分析与信息收集作为支撑能力

为什么重来？

旧项目三个致命伤：

1. **Prompt 驱动而非代码驱动** — Markdown 文件堆砌，无法测试、无法版本管理
2. **没有质量门禁** — AI 产出直接进知识库，错误静默累积
3. **文件系统混乱** — prompt、工具、知识、状态全揉在一个目录

这次怎么做？

一个词：**架构优先**。过去两天（5月20-21日）完成了：

- 22 项架构决策全部确认
 - 设计规格通过 2 轮 × 6 位 AI 审查者审查（51个问题全部修复）
 - 实施计划正在编写（5个阶段，~8,200 LOC）
-

二、架构概要（5分钟）：系统的骨架

核心理念

你（人类）→ 给目标 → Supervisor Agent 分解任务 → 分配给子 Agent 执行
→ 汇总 Findings → 你审核

不是你手动操作工具，而是你告诉系统"分析这个磁盘镜像有没有数据泄露的痕迹"，系统自动分解、调度、汇总。

六个设计原则（记住这六条就够）

#	原则	一句话
1	**抽象优先，避免锁定**	每层都有接口+至少两种实现，离线/在线无缝切换
2	**程序做搬运，AI做判断**	Syncthing 传文件，AI 只读写本地；绝不写 HTTP 调用
3	**置信度分级**	所有输出标注 ✅ 已验证 / ⚠️ 推测 / ❌ 未知
4	**模块自治，核心统一**	每个插件独立开发，核心只定义接口契约
5	**知识是一等资产**	每个案件自动汇入知识库，越用越强
6	**自我改进是基础设施**	MetaAgent 7×24 巡检，发现新工具/新技术/新方法，人类审批后自动升级

技术栈

- **语言:** Python 3.12（主体）+ Rust/PyO3（性能热点）
- **LLM:** 统一抽象层，支持 OpenAI / Anthropic / DeepSeek / Ollama / vLLM
- **数据:** PostgreSQL（历史查询）+ 本地 YAML 文件（实时协作）
- **协作:** Syncthing（P2P 文件同步）+ Git（代码）
- **隔离:** Docker（常规任务）+ Firecracker microVM（高危任务）

三、团队模型（5分钟）：Cell 制

不是部门制，是 Cell 制

传统部门制：
部门长 → 成员1，成员2，成员3

Cell 制：
每个人 + AI = 独立生产力单元
Cell owner 管自己的 AI 干活

核心理念：成员当小组长，AI 是手下员工。你不是来写代码的——你是来指挥 AI 写代码的。

仓库分工 (GitHub Org: forhacker)

仓库	需要的人	角色
`core/`	**你 + 2位师兄**	核心框架、接口契约、CI 模板。这是心脏，必须最稳
`plugin-forensics/`	**2位竞赛老手**	取证插件（内存分析、磁盘分析、文件分析）
`plugin-pentest/`	**1老手 + 1新手**	渗透测试插件
`plugin-osint/`	**1中级 + 1新手**	开源情报插件
`plugin-ctf/`	**1新手 + AI**	CTF 专项插件（门槛最低，适合练手）
`knowledge-base/`	**自动化(CI)**	共享知识库，代码自动汇入，无需人力
`docs/`	**集体维护**	中央文档

三种贡献路径

水平	人数	初始角色	成长路径
有基础	~5人	插件 owner，管理 AI 开发	→ 核心贡献、架构设计
有想法	~2人	核心贡献 + Code Review 辅助	→ 框架设计、技术决策
零基础	~3人	测试、文档、报告、跟 AI 学	→ 简单插件 → 独立 Cell owner

四、第一个迭代（10分钟）：Phase 1 任务分解

Phase 1 概览：Core Skeleton（第1-3周，~2,000 LOC）

目标: 搭好骨架，跑通第一条端到端流程——"用 CLI 分析一个内存镜像"。

具体任务（按顺序）

Task 1: 项目脚手架 [今天就能开始]

做什么: `pyproject.toml`、目录结构、`init.py`、`tests/conftest.py`

谁来做: 你或师兄，30分钟搞定

产出: 可以 `pip install -e .` 的空项目

Task 2: LLM 抽象层 [核心，需要理解抽象]

做什么: `LLMBackend` ABC、`Message` / `LLMResponse` 类型

谁来做: 你 + 1位有思想的师兄

关键: 这是整个系统的基石——所有 Agent 通过这个接口调 LLM

测试: 5个单元测试（抽象类不可实例化、`Message`字段、`Response`字段等）

Task 3: OpenAI + Ollama 后端实现 [并行]

做什么: 对接 OpenAI API 和本地 Ollama

谁来做: 1位有Python基础的人 + AI

关键: Ollama 用 OpenAI 兼容接口，代码可以很薄

Task 4: DeepSeek + Anthropic + 路由 [并行]

做什么: DeepSeek（继承OpenAI）、Anthropic（独立实现）、`SensitivityRouter`

谁来做: 1位有基础的人 + AI

关键: 路由逻辑：高敏感→本地，低敏感→云端，`FORHACKER_OFFLINE=1` 全局覆盖

Task 5: MessageBus [1天]

做什么: `MessageBus` ABC + `InProcessBus` (`asyncio.Queue`)

谁来做: 任何人 + AI

关键: 默认进程内零依赖，分布式版本后续扩展

Task 6: 插件系统 [2天]

做什么: `BasePlugin` ABC、`PluginManager` (发现+加载+故障隔离)、`CapabilityRegistry`

谁来做: 1位有经验的 + 需要理解插件架构

关键: 这是所有插件 Cell 的基础设施

Task 7: PostgreSQL 数据层 [2天, 可并行]

做什么: SQLAlchemy models、数据库迁移、连接管理

谁来做: 1位了解数据库的人

Task 8: CLI 入口 [2天]

做什么: click typer CLI (`forhacker case create` , `forhacker task run`)

谁来做: 任何人 + AI (click 很简单)

Task 9: 端到端集成测试 [3天]

做什么: 搭一条完整链: CLI → LLM → 工具调用 → Finding 写入

谁来做: 你 + 师兄主导

关键: 这是 Phase 1 的验收标准——跑通就证明骨架对了

分工原则

1. Task 2-3-4 串行依赖, 必须按顺序做 (先定义 ABC, 再实现后端, 最后路由)
2. Task 5-6-7-8 可以并行
3. 零基础成员从 Task 5 和 Task 8 开始
4. 你控 Task 1、2、9 三个关键节点
5. 每个 Task 都是"人类决策 + AI 干活"模式——成员主要负责想清楚需求、审查 AI 产出

五、质量铁闸 (3分钟) : 没有例外

每个 PR 必须通过四道门:

- | | |
|-------------------------------------|---|
| 1. <code>ruff format --check</code> | ← 格式 |
| 2. <code>mypy src/</code> | ← 类型检查 |
| 3. <code>pytest --cov≥70%</code> | ← 测试覆盖率 |
| 4. AI Code Review | ← AI 审查 (我们自己的 <code>superpowers skill</code>) |

- CI 自动执行，不伤感情
- 零基础成员在报错中学习——这是设计如此
- 你和其他核心成员只做**策略审核**（架构对不对，不是语法对不对）

六、协作基础设施（2分钟）

工具	用途
GitHub	代码托管 + PR 审查 + CI/CD
Syncthing	`shared/` 目录实时同步（任务状态、Findings、答案汇总）
Chrome Remote Desktop	手机远程操控电脑（先过渡用）
PostgreSQL	案件元数据、审计日志、历史查询

关键规则：**AI 绝不写 HTTP 调用做协作。AI 只读写本地文件，Syncthing 负责搬运。**

七、时间线（2分钟）

阶段	时间	关键交付
Phase 1: 骨架	第1-3周	LLM层、插件系统、CLI、第一条端到端流程
Phase 2: Supervisor	第4-6周	DAG引擎、Supervisor调度、首个取证插件
Phase 3: 协作	第7-9周	Syncthing集成、CI模板、知识库自动汇入
Phase 4: MetaAgent	第10-12周	四角色自进化、Web Dashboard、审计回滚
Phase 5: 打磨	第13周+	插件市场、脚手架CLI、沙箱隔离

本周目标: 完成 Task 1-2-3（项目骨架 + LLM 抽象层 + OpenAI/Ollama 后端），所有人配好开发环境。

八、你需要他们今天带走的三件事

1. 安装环境（今天回去就做）

```
# 安装 uv (Python 包管理器)
pip install uv

# 装 Python 3.12
uv python install 3.12

# 装 Ollama (本地模型)
# 下载: https://ollama.com
ollama pull llama3.2:3b # 先拉个小模型测试

# 装 Docker Desktop
# 下载: https://docker.com

# 装 Syncthing
# 下载: https://syncthing.net
```

2. 认领方向（今天确定）

每个人选一个自己感兴趣的方向：

- 取证（内存/磁盘/文件）
- 渗透测试（扫描/exp利用/后渗透）
- 开源情报（社交媒体/域名/暗网）
- CTF（既然是警院，CTF 是最自然的起点）

3. 阅读材料（这周内）

- **必读：** docs/superpowers/specs/2026-05-21-forhacker-design.md （设计规格）
- **必读：** 记录/设计日志.html （设计决策可视化版，浏览器打开）
- **推荐：** 学习怎么用 Claude Code / Cursor 让 AI 写代码（这是我们所有人的基础技能）

九、可能会被问到的问题（准备回答）

Q: 我是零基础，能做什么？

A: 先从测试和文档开始。测试是学代码最快的方式——你看测试就知道每个模块应该怎么用。我们每个模块都有 TUTORIAL.md。你有 AI 当私教，不会就问。

Q: 我们和直接用 ChatGPT/Claude 写工具有什么区别？

A: 区别在于系统化和积累。你每次用 ChatGPT 都是一次性的——下次还得重新描述。forhacker 把每次分析的结果自动存进知识库，第二次遇到类似问题直接调取。而且 Supervisor 能自动分解复杂任务，你不用手动把大问题拆成小问题。

Q: 这个项目怎么参加比赛？

A: 线上赛直接用——Supervisor 集中调度，Syncthing 同步答案。线下赛（断网）用本地模型（Ollama），功能完整。你们在 CTF 中用这个平台本身就是最好的测试。

Q: 服务器 2C8G 20G 够用吗？

A: 只跑 PostgreSQL 和 Syncthing 中继——够了。检材在各自电脑上，几百 GB 的镜像不存服务器。后续有经费再扩展。

十、你的行动清单（会议后）

1. ☐ 创建 GitHub Organization `forhacker`
2. ☐ 创建 `core/` repo，初始提交 `pyproject.toml` + 目录结构
3. ☐ 给每个人发 GitHub 邀请
4. ☐ 建微信群/企业微信群，日常异步沟通
5. ☐ 每个人报自己感兴趣的方向和当前水平（Google Form 或接龙）
6. ☐ 约下次同步时间（建议3天后，检查环境安装进展）
7. ☐ 把 `记录/设计日志.html` 发到群里（浏览器就能看，最直观）

这次会议的核心信息：**我们不写代码，我们指挥 AI 写代码。我们要建的是一套让 AI 协作取证的系统，不是一个脚本集合。每个人管自己的 Cell，核心组管接口契约。**